LLM エージェントの履歴要約と終了制御による実行費用の評価

桐生　篤希[†a)]（学生員）
菊澤百々菜[†]（学生員）
木村　秀明[†]（正員：フェロー）

Evaluating Execution Cost of History Summarization and Termination Control for LLM Agents

Atsuki KIRYU[†a)], Momona KIKUZAWA[†], *Student Members*, and Hideaki KIMURA[†], *Fellow*

[†] 中部大学大学院工学研究科情報工学専攻，愛知県
Department of Computer Science, Graduate School of Engineering, Chubu University, 1200 Matsumoto-cho, Aichi, 487-0027 Japan
a) E-mail: tp26013-0852@sti.chubu.ac.jp
DOI:10.14923/transj.???????????

あらまし　LLM エージェントの履歴要約方式と API 実行記録に基づく終了制御を独立に評価する．要約生成を含む全モデル呼出しを計測し，今回の既知課題で成功を保ちながら総トークン数が減少したことを確認した．呼出し配置と完了後の生成に着目した比較結果を示す．

キーワード　LLM エージェント，ハーネス，履歴要約，終了制御，トークン使用量

1. はじめに

日常業務のデータ整理や事務処理を支援する大規模言語モデル（LLM）の研究が進んでいる．写真整理や精算では，文章生成に加え，外部アプリケーションで対象の検索，状態の変更，結果の確認を連続して行う必要がある．推論と行動を繰り返す ReAct [1] や，コードを行動として実行する CodeAct [2] は，こうした処理を支える．

しかし，実行が長期化すると，過去の操作や観測結果を次の生成へ繰り返し入力する費用が増大する．また，作業が完了した後もエージェントが確認や生成を続ければ，成果を増やさずに計算資源を消費し得る．単純に履歴を短縮するだけでは課題の遂行に必要な情報を失うおそれがあり，一方で早期に終了させれば課題が未完了に終わるおそれがある．そのため，課題の最終的な成功と実行全体の費用を同時に検証する必要がある．

そこで本稿では，モデルの再学習を伴わずに実行を管理するハーネスを用い，履歴要約の呼出し配置と終了制御を評価する．履歴要約では，要約を生成する時点，行動生成と要約の呼出しを分けるかどうか，要約機会の上限を比較軸とする．また，終了制御では，実行中に取得できる API の要求と応答を記録し，この API 実行記録に基づく制御を通常継続と比較する．両者を独立した実験として実施し，要約生成を含む全モデル呼出しの総トークン数と，課題の最終的な成功を調べる．

2. 関連研究

入力圧縮の分野では，長文中の情報の位置が性能に影響すること [3] を踏まえ，LLMLingua [4] が予算制御とトークン選別を，LLMLingua-2 [5] が蒸留データによるトークン分類を，LongLLMLingua [6] が長文の重要情報を残す圧縮をそれぞれ扱っている．

長期実行の観点では，ACON [7] が観測・履歴の圧縮方針を，TokenPilot [8] が入力圧縮とキャッシュ継続性を検討している．さらに，Generative Agents [9] は経験の記憶・抽象化・検索を，Reflexion [10] は言語的フィードバックを後続試行に利用する手法を扱っている．本稿では，要約専用の呼出しを用いる方式と，行動と同じ応答内に含める方式を比較し，要約生成も含めた課題全体のトークン使用量を評価する．

一方，終了制御に関しては，Runaway [11] が終了指示と外部の完了検証を検討している．本稿では，既知の要求形式について API 実行記録から完了条件を検査する．通常継続との比較を通じて，課題の最終的な成功，API 要求数，総トークン数を評価する．

3. 提案手法

ハーネスとは，モデルの外側で入力と実行の継続を管理するプログラムを指す．本稿では，既存の要約を用いて入力履歴を更新する処理と，API 実行記録に基づいて次の生成の要否を判断する処理をそれぞれ構成する．図 1 の実験 A に要約と行動生成の呼出し配置を，実験 B に完了判定に基づく終了制御を示す．両者は個別に評価し，組合せによる削減効果は本稿の対象外とする．

専用 1 回方式と専用反復方式では，行動と要約の呼出しを分ける構成を共通とし，要約機会の上限を比較する．専用反復方式と行動併記方式では，機会の上限を揃え，要約候補を専用呼出しで生成するか，行動と同じ応答に含めるかを比較する．候補の採否と入力の更新は，いずれもモデルの外側のハーネスが担う．

まず，課題と履歴からモデルが行動を生成し，ハーネスがプログラムを実行して行動と結果を記録する．本稿では，行動生成から実行結果の記録までを 1 ターンと定義する．

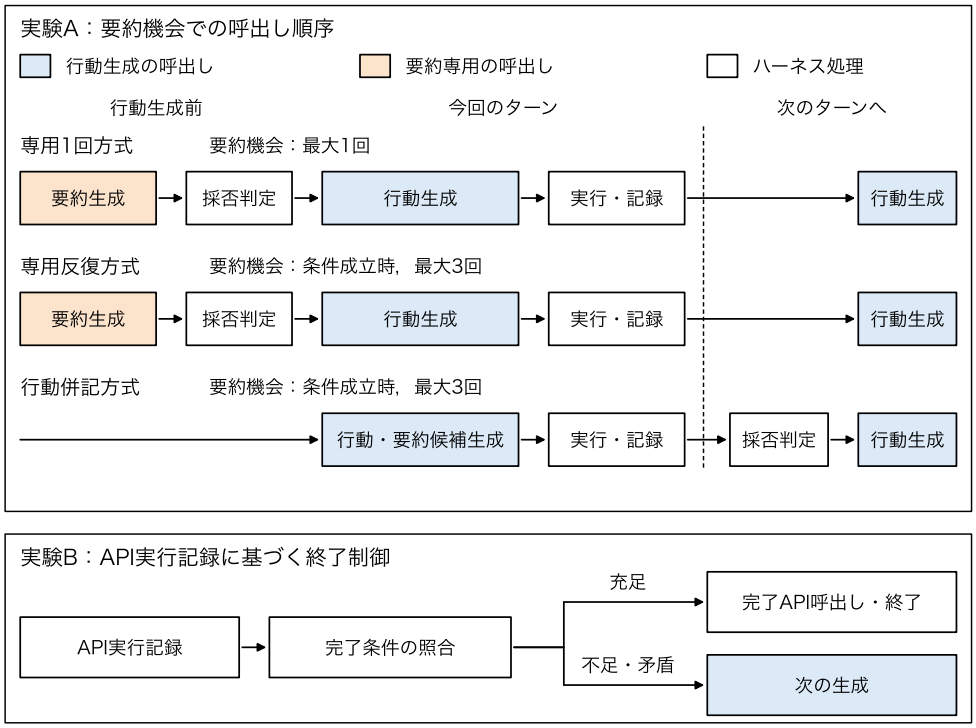


図 1　履歴要約方式と終了制御

3.1　入力履歴を短くする制御

履歴の蓄積による入力の長大化を抑えるため，保存した全履歴は維持しつつ，モデルへ渡す古い履歴を要約に置き換える．生成された要約のうち，検査を経て採用される前のものを要約候補と呼ぶ．本稿では，要約の生成のみを目的とするモデル呼出しを専用呼出しと定義し，以下の 3 方式に比較用の名称を付ける．

専用 1 回方式は，要約専用のモデル呼出しを最大 1 回追加する方式である．入力長が閾値に達し，かつ要約可能な古い履歴が存在する最初の機会に候補を生成する．候補を検査して採用した場合は，要約と未要約の履歴を次の行動生成へ渡す．要約機会は最大 1 回とし，その機会が終了した後は新たな要約を試みない．

専用反復方式は，要約専用の呼出しを必要に応じて繰り返す方式である．最初の要約機会では専用 1 回方式と同じ処理を行う．それ以降は，入力長と新たに完了したターン数の条件を満たすたびに，その時点で未要約の古い履歴を要約対象とする．採用済みの要約が存在する場合は，それも新たな要約対象に含める．要約機会は最大 3 回とし，各機会で候補の採否を決定してから次の行動生成へ進む．

行動併記方式は，行動を生成するモデルの同じ応答内に，行動と要約候補を出力させる方式である．要約専用の呼出しは追加せず，最大 3 回の要約機会を設ける．この方式では，応答内の全プログラムを実行して結果を記録した後，次の行動生成へ進む前に候補の採否を決定する．ただし，候補には同じ応答で生成した行動の実行結果を含めない．その行動と結果は原文のまま次の入力に残す．

どの方式でも，要約には完了した作業，次の操作に使う情報，残作業，不確かな点の四つの欄を設ける．要約候補が得られた際，ハーネスはまず形式と文字数を検査する．さらに，元の指示と課題，要約，未要約の履歴を含めた次回入力全体のトークン数を算出し，候補を採用しない場合より厳密に少なくなることを確認して採用する．要約候補を生成する時点では，直近 2 ターンを原文のまま保持する．ここで検査するのは構造と入力長であり，要約内容の正確性を保証するものではない．

具体的には，完了した作業の欄には実行済みの内容を，次の操作に使う情報の欄には識別子や API の呼出し方，保持された変数などを残すよう指示する．一方，残作業と不確かな点の欄では，未実施の操作と未確認

の事項を区別させる．また，API の正常応答だけを根拠に課題の完了を推定しないよう求める．

元の指示と課題，直近の行動と API 応答は原文で保持し，要約後も要求内容と直近の実行結果を参照できるようにする．

採用条件を満たさなかった場合は，それまでの要約と未要約の履歴を維持する．まだ一度も要約を採用していない段階では，要約前の履歴をそのまま使い続ける．要約機会とは候補の生成を試みる機会を指し，採用回数とは区別する．不採用の場合も 1 回の機会として数え，その場での再生成は行わない．専用 1 回方式は機会の上限を定める名称であり，生成や採用が 0 回となる場合もある．入力長とターン数の閾値は 4.1 に示す．保存した全履歴は，入力を短縮した後も別に保持する．

専用呼出しを用いる方式では，候補生成に 1 回のモデル呼出しを要する．一方，行動併記方式では行動生成の呼出しを共有する．採用回数は，候補を採用して入力を実際に更新した回数を指す．本稿では，要約機会数，採用回数，モデル呼出し数を明確に区別して扱う．また，要約の採用後も，新たな行動と結果は未要約の履歴として追加され，次の要約機会まで入力に残る．

3.2 作業完了後の生成を省く制御

終了制御とは，API 実行記録から課題の完了状況を検査し，根拠が揃った時点で次の生成を省く処理である．本稿では，この検査段階を完了判定と呼ぶ．写真を作成時期に応じた指定先へ移す課題を対象とし，既知の一つの要求形式に対応する検査規則を手書きで用意する．まず，モデル応答内の全プログラムを実行し，API の要求と応答を記録する．次に，移動前の写真一覧，各写真の作成日時，API から取得した現在年を確認する．これらの情報に基づき，全ての対象写真が指定先へ移動し，元の名前が保たれているかを照合する．完了の根拠が初めて揃った時点で完了 API を 1 回呼び出し，次の生成を省く．

この完了判定はモデル自身の完了宣言に依存せず，API 実行記録に対象全件の処理を確認できる情報が含まれているかを調べる．比較対象となる通常継続では，この判定による終了を行わず，通常の実行手順を続ける．

一部の写真の移動だけで完了と判断しないよう，移動前に対象集合を確定する．作成日時と API から得た現在年から移動先を照合し，名前の保持も別に確認する．根拠は API 実行記録から取得し，モデルの推測や計算機の時刻では補わない．

対応する要求形式から外れる場合や，判定に必要な記録が不足・矛盾している場合には，通常どおり次の生成へ進む．判定には，その時点で得られた API 実行記録のみを使用し，公式評価器の正解は利用しない．履歴要約との組合せ効果は評価しない．

3.3 総トークン数と課題の成功

1 回あたりの入力を短縮しても，要約生成やその後の行動生成に使用するトークンが増えれば，課題全体の使用量は増加し得る．そのため，要約後の入力長だけを比較せず，課題開始から終了までを全方式に共通する集計範囲とする．

条件 p で実行したモデル呼出しの集合を C_p とする．各呼出し c の入力トークン数を I_c，出力トークン数を O_c とし，実行全体の総トークン数 T_p を式 (1) で求める．

$$T_p = \sum_{c \in C_p} (I_c + O_c) \tag{1}$$

要約処理自体にも生成費用がかかるため，行動生成と要約生成の両方を合算して集計する．不採用となった要約候補の生成や，失敗時の呼出しについても，取得できた使用量は全て計上する．使用量が不明な場合は 0 とせず，既知分の小計と不明件数を記録する．

課題の成功は，実行終了後の最終状態に対して AppWorld [12] の公式評価器を用いて判定する．評価器の正解や詳細情報は，実行中のモデルへ返さない．方式間では成功数に加え，各課題の成功が維持されているかを確認する．

同じ課題を処理した方式同士で成功と総トークン数を対応させる．課題をまたぐ合計では減少と増加が相殺されるため，実験 A では課題別の値も示す．

本指標はキャッシュされた入力も含む論理トークン数であり，課金額や実行時間を直接表さない [8]．要約生成を含む費用評価は ACON [7] にも用いられている．履歴要約は次回入力の長さを，終了制御は次の生成の要否を管理するため，実験 A と B は独立に集計・比較する．

4. 実験 A：要約の呼出し配置

4.1 条　　件

写真操作，精算，返金，CSV 処理の既知 4 課題に対し，専用 1 回方式，専用反復方式，行動併記方式の各方式を各課題で 1 回ずつ実行した．4 課題に 3 方式を適用し，計 12 実行とした．モデルは全条件で

MiniMax-M3 を用い，GGUF 形式の IQ4_XS 量子化モデルを llama.cpp b11669 で実行した．文脈長は 65,536 トークンである．乱数の初期値を表す seed は，モデル生成の基点を 57，環境を 42 とした．生成側は呼出しの役割と順序に従い seed を増分した．

要約機会を設ける入力長の閾値は 16,384 トークンとし，要約時には直近 2 ターンを原文のまま保持する．専用反復方式と行動併記方式では，前回の要約機会が終了してから新たに 3 ターン以上が完了することを要求する．採用と不採用のどちらでも，その機会の終了時点を次回の起点とする．初回の起点は課題開始である．要約は合計 8,192 文字以下とし，次の行動生成へ渡す入力全体が短くなる場合に採用する．要約専用の呼出しでは出力上限を 4,096 トークンとし，その使用量も式 (1) へ含める．

各実行の上限はモデル呼出し 80 回，総トークン数 450,000，作業時間 1,800 秒とし，環境 API は写真 160，他 3 課題 80 要求とした．生成の温度は行動生成で 1.0，要約専用の生成で 0.7 とした．並列実行数は 1，KV キャッシュは q8_0，計算機は Apple M3 Ultra，メモリは 256 GiB である．実装と記録を公開する(注1)．

4.2 全 4 課題の結果

表 1 に 4 課題の結果を示す．全 12 実行で課題の成功を確認し，使用量の欠測はなかった．

表 1　実験 A の課題別総トークン数

課題	専用 1 回方式	専用反復方式	行動併記方式
写真操作	139,871	139,871	129,374
精算	211,110	211,110	211,110
返金	102,087	102,087	102,087
CSV 処理	440,461	332,588	394,351
合計	893,529	785,656	836,922

要約生成を含む 4 課題の合計では，専用反復方式が 785,656 トークンで 3 方式中最少となった．専用 1 回方式の 893,529 トークンと比較して 107,873 トークン少なく，削減率は 12.07% であった．ただし，この差は CSV 処理のみで生じており，残る 3 課題では両方式の値が一致した．

写真操作では，専用 1 回方式と専用反復方式が 139,871 トークンであったのに対し，行動併記方式は 129,374 トークンとなった．行動併記方式は他の 2 方式より 10,497 トークン少なかった．要約の採用回数はいずれも 1 回であったが，同じ採用回数でも課題全体の総トークン数には差が生じた．

一方，精算の総トークン数は全方式で 211,110 トークンであった．要約の採用回数はいずれも 0 回であり，方式による総量の差は見られなかった．

返金では，全方式で 102,087 トークンを使用し，要約の採用回数は 0 回であった．精算と同様に，成功と総トークン数は全方式で一致した．

CSV 処理では，専用 1 回方式が 440,461，専用反復方式が 332,588，行動併記方式が 394,351 トークンであった．専用反復方式は専用 1 回方式より 107,873 トークン少なく，4 課題合計での削減はこの課題で生じた．要約の採用回数は専用 1 回方式が 1 回，専用反復方式と行動併記方式が各 2 回であった．専用反復方式と行動併記方式は採用回数が等しかったものの，総トークン数は一致しなかった．

行動併記方式は，4 課題合計で専用 1 回方式より 6.34% 少なかったものの，専用反復方式より 51,266 トークン多く，増加率は 6.53% であった．専用反復方式との比較では，写真操作で 10,497 トークン減少した一方，CSV 処理で 61,763 トークン増加した．今回の 4 課題では，専用反復方式が総トークン数の合計で 3 方式中最少となった．

5. 実験 B：API 実行記録に基づく終了制御

5.1 条　　件

同じ写真課題について，API 文書の取得を必須とする文書必須条件と，公開 API 名からのアクセスを許す名前参照条件を設けた．各条件で，通常継続と，API 実行記録に基づく終了制御を比較した．終了制御は，事前に実行・保存した通常継続と同じ初期状態，モデル，seed，モデルへの指示とツール説明，予算を用いて各 1 回実行した．

seed の基点は 42 であり，実験 A とは別の履歴・設定である．同じ MiniMax-M3 IQ4_XS と llama.cpp b11669 を用い，各実行の予算はモデル呼出し 80 回，450,000 トークン，1,800 秒，API160 要求とした．

完了判定と完了通知は 3.2 に従う．判定，記録保存，完了通知を含む実行時間を記録し，完了通知を API 要求数に含めた．

行動の生成は LLM が担い，記録の検査はハーネスの規則で行う．この検査のために別のモデル呼出しは設けない．完了の根拠が揃えば次の行動生成を省き，根拠が足りなければ同じ実行を継続する．

(注1)：https://github.com/sophiria1225/AI-

終了以前の生成過程の違いが比較に含まれていないか確認するため，終了制御側で実行したモデル呼出しの要求と応答が，通常継続側の先頭から同じ範囲と完全に一致することを事前の照合基準とした．API 操作の記録と最終状態も比較した．

5.2 成功を保った総トークン数の削減

表 2 に結果を示す．両 API 条件で，終了制御を用いた実行は課題の成功を保ち，通常継続より総トークン数が少なかった．文書必須条件では 130,439 から 91,116 へ 30.15%，名前参照条件では 151,664 から 112,233 へ 26.00% 減少した．全 4 実行で公式評価器による成功を確認し，使用量の欠測はなかった．各条件で，モデル呼出しは 2 回，API 要求は 4 回減少した．完了通知を含めて処理を終えた実行では，移動後の確認 API4 回と，それに伴う生成が省かれた．

表 2　実験 B の総トークン数と呼出し回数

API 条件	終了方法	総トークン数	モデル呼出し数	API 要求数
文書必須	通常継続	130,439	15	104
文書必須	終了制御	91,116	13	100
名前参照	通常継続	151,664	14	103
名前参照	終了制御	112,233	12	99

実行した API 操作などの前半の記録と，実行終了時の 13 個のデータベースのハッシュ値は通常継続と一致した．今回の比較では，最終状態と課題の成功を保った総トークン数の減少を確認した．

6. 考　　察

実験 A では，要約専用呼出しを省く行動併記方式より，専用反復方式の合計が少なかった．方式選択では，要約生成も含む課題全体の使用量を比べる必要がある．実験 B の結果は，API 実行記録に基づく完了判定を用い，作業後の確認と生成を省く実行管理の有用性を示唆する．

本評価は既知課題，単一モデル・量子化設定，少数の seed・試行による開発実験の事後整理である．実験 B の各 1 回の比較では，要求・モデル応答全文の完全一致基準が不成立であり，観測差には生成過程の違いも含まれ得る．平均的な因果効果と汎用的な終了保証，未見課題での品質維持と統計的非劣性の検証を今後の課題とする．

7. む す び

本稿では，ハーネスによる履歴要約と終了制御を独立に評価し，既知課題で成功を保った総トークン数の削減を観測した．今後は，対象課題と要求形式を広げ，複数の seed による評価を通じて，多様な作業に対応する実行管理技術へ発展させていく予定である．

文　　献

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” Proc. ICLR, May 2023.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” Proc. ICML, PMLR, vol.235, pp.50208–50232, July 2024.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” Trans. ACL, vol.12, pp.157–173, 2024. doi: 10.1162/tacl_a_00638.

[4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” Proc. EMNLP, pp.13358–13376, Dec. 2023. doi: 10.18653/v1/2023.emnlp-main.825.

[5] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” Findings of ACL: ACL 2024, pp.963–981, Aug. 2024. doi: 10.18653/v1/2024.findings-acl.57.

[6] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” Proc. ACL, vol.1, pp.1658–1677, Aug. 2024. doi: 10.18653/v1/2024.acl-long.91.

[7] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing context compression for long-horizon LLM agents,” Proc. ICML, PMLR, vol.306, pp.55321–55375, July 2026.

[8] B. Xu, Z. Xue, D. Chen, C. Fu, C. Wu, C. Huang, C. Jiang, J. Fang, X. Deng, Y. Chen, Y. Yao, X. Wang, J. Shang, H. Chen, and N. Zhang, “TokenPilot: Cache-efficient context management for LLM agents,” Findings of ACL: EMNLP 2026, pp.19034–19049, Oct. 2026.

[9] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” Proc. ACM UIST, article 2, pp.1–22, Oct. 2023. doi: 10.1145/3586183.3606763.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” Advances in Neural Information Processing Systems, vol.36, pp.8634–8652, 2023. doi: 10.52202/075280-0377.

[11] Q. Lu, L. Ding, S. Cao, X. Liu, K. Zhang, J. Zhang, and D. Tao, “Runaway is ashamed, but helpful: On the early-exit behavior of large language model-based agents in embodied environments,” Findings of ACL: EMNLP 2025, pp.24014–24027, Nov. 2025. doi: 10.18653/v1/2025.findings-emnlp.1304.

[12] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” Proc. ACL, vol.1, pp.16022–16076, Aug. 2024. doi: 10.18653/v1/2024.acl-long.850.

（xxxx 年 xx 月 xx 日受付）

Abstract We independently evaluated history summarization methods and termination control based on API execution records for LLM agents. Counting all model calls, including dedicated summarization, showed reductions in total tokens while preserving success on known tasks under the evaluated conditions.

Key words LLM agents, harness, history summarization, termination control, token usage